Homework 2

Due : October 17th at 11 : 59 pm

Deliverables. Submit a single PDF of your write-up to Gradescope *HW2 Written Questions*.

Start each problem on a new page.

Honor Code

Write and sign the following statement:

> "I certify that all solutions in this document are entirely my own and that I have not looked at anyone else's solution. I have given credit to all external sources I consulted."

> Since time is valuable and this is a non-credit independent study, most of the typesetting and English translation here were produced by Cursor based on my handwritten scripts. I'm grateful for the help that AI has provided to my learning.

*Written By Ziyu Li on Dec*22, 2025

1 Sum of Residuals

Consider a dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ with $x_n \in \mathbb{R}^{D+1}$ and $y_n \in \mathbb{R}$. Here we assume that X already contains a column of 1s. We model the relationship between x and y using linear regression with parameters $w \in \mathbb{R}^{D+1}$:

$$\hat{y} = x^\top w.$$

We have computed the solution w^* to the least squares regression problem:

$$w^* = \arg\min_w \frac{1}{2} \sum_{n=1}^N \left(y_n - x_n^\top w\right)^2.$$

The residual vector is then defined as:

$$r = y - Xw^*$$

(a) (*2 pts*) Use the normal equation to show that: $X^\top r = 0$.

The objective can be written as $\frac{1}{2}\|y - Xw\|_2^2$. Taking gradient w.r.t. w and setting it to zero gives the normal equation:

$$\nabla_w \tfrac{1}{2}\|y - Xw\|_2^2 = -X^\top (y - Xw) = 0.$$

Evaluating at w^* yields

$$X^\top y = X^\top X w^*.$$

Rearranging and using $r = y - Xw^*$ gives

$$X^\top r = X^\top y - X^\top X w^* = 0.$$

(b) (*2 pts*) Use the presence of a bias term (encoded in the column of 1s in X) to show that the sum of the residuals is 0.

Let $\mathbf{1} \in \mathbb{R}^N$ be the all-ones vector. Because X contains a column equal to $\mathbf{1}$, one column of $X^\top r$ is $\mathbf{1}^\top r$. From part (a) we have $X^\top r = 0$, so in particular

$$\mathbf{1}^\top r = 0.$$

Therefore the sum of the residuals $\sum_{n=1}^N r_n = \mathbf{1}^\top r$ equals 0.

(c) (*2 pts*) Suppose we add a point (x_j, y_j) to our dataset with a very large $|y_j|$ but x_j is close to the mean of x. Predict qualitatively how w will change and explain why least squares is sensitive to such a point. How can you modify the loss to make the model more robust to outliers?

If we add a point (x_j, y_j) with very large $|y_j|$ while x_j is near the mean of the x's, the fitted parameter w will typically shift its intercept (bias) to reduce that large residual, while slope components change less when x_j has low leverage. Least squares is sensitive to such a point because the squared loss penalizes large residuals quadratically, so an outlier with large residual produces a disproportionately large contribution to the loss and gradient.

To make the model more robust to outliers one can:

- Use a robust loss, e.g. absolute error (L1), Huber loss, or Tukey's bisquare, which reduce the influence of large residuals.

- Use weighted least squares or iterative reweighting that downweights points with large residuals.
- Use robust estimators like RANSAC, Theil–Sen, or M-estimators.
- Note: regularization (e.g. ridge) stabilizes coefficients but does not remove sensitivity to large y outliers; combine regularization with a robust loss if needed.

2 Weighted Linear Regression

Consider a dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ with $x_n \in \mathbb{R}^{D+1}$ and $y_n \in \mathbb{R}$. Here we assume that X already contains a column of 1s. We model the relationship between x and y using linear regression with parameters $w \in \mathbb{R}^{D+1}$:

$$\hat{y} = x^\top w.$$

Suppose we have nonnegative weights $\{\alpha_n\}_{n=1}^N \geq 0$ for each observation and our goal is to minimize the weighted least squares objective:

$$L(w) \;=\; \sum_{i=1}^{n} \alpha_i (y_i - x_i^\top w)^2.$$

In this problem, we will derive the w^* that minimizes this loss.

(a) (*2 pts*) Write the weighted least-squares loss function $L(w)$ in matrix form using the diagonal matrix:

$$A = \mathrm{diag}(\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^{N\times N}.$$

Your answer, should not contain any summations.

The weighted least-squares loss in matrix form is

$$L(w) = (y - Xw)^\top A (y - Xw).$$

(b) (*2 pts*) Compute the gradient of the loss and express your answer in matrix form (no summations). You can do this by working with the summation form and taking partial derivatives or by using the following gradient identities:

$$\nabla_w \left(u^\top w\right) = u$$
$$\nabla_w \left(w^\top A w\right) = (A + A^\top) w$$

Since A is diagonal (hence symmetric), the gradient is

$$\nabla_w L(w) = -2X^\top A(y - Xw) = 2X^\top AXw - 2X^\top Ay.$$

(c) (*3 pts*) Set the gradient equal to zero, and solve for the closed-form solution for w^* that minimizes the weighted least squares objective. in terms of X, y, and A, assuming $X^\top AX$ is invertible.

Setting $\nabla_w L(w) = 0$ gives the weighted normal equations

$$X^\top AX\, w = X^\top Ay.$$

Hence the closed-form minimizer is

$$w^* = (X^\top AX)^{-1} X^\top Ay.$$

(d) (*3 pts*) Under what condition is $X^\top AX$ guaranteed to be invertible? Can we derive a simpler condition when $\alpha_i > 0$?

In general $X^\top AX$ is invertible iff for all nonzero $v \in \mathbb{R}^{D+1}$,

$$v^\top X^\top AXv = \sum_{i=1}^{N} \alpha_i (x_i^\top v)^2 \neq 0,$$

i.e. the only v satisfying $x_i^\top v = 0$ for every index with $\alpha_i > 0$ is $v = 0$.

If all weights satisfy $\alpha_i > 0$, then A is positive definite and the condition simplifies: $X^\top AX$ is invertible if and only if X has full column rank (rank $= D + 1$), i.e. the columns of X are linearly independent.

3 Online K-Means

Consider the sequential (online) version of the K-means algorithm. Online algorithms assume that you receive data points in a *stream* rather than having all of your data already stored.

In online K-means, instead of using the full dataset to update cluster centers at each iteration, we update the nearest prototype μ_k for each data point x_n as it is observed, using the rule:

$$\mu_k \leftarrow \mu_k + \frac{1}{N_k}(x_n - \mu_k),$$

where N_k is the number of points that have been assigned to cluster k so far. Each point is assigned to the cluster with the nearest center at the time it is observed.

(a) (*3 pts*) Explain how this sequential update differs from batch K-means in terms of memory and computation. *Hint: How often is each data point used in regular K-means compared to online K-means?*

The main differences between the sequential (online) update and batch K-means are:

- **Memory:** Batch K-means requires storing and accessing the entire dataset each iteration, with memory roughly $O(ND)$. Online K-means only needs to store the cluster centers and counts, with memory roughly $O(KD)$, independent of N, making it suitable for streaming or very large datasets.
- **Computation:** Batch K-means computes distances from all N points to K centers each iteration (complexity per iteration about $O(NKD)$) and typically requires multiple passes over the data. Online K-means processes each arriving point once (or a few times) performing one assignment and one incremental update, with per-point cost about $O(KD)$.
- **Data usage frequency:** In batch K-means each data point is reused in every iteration (multiple passes); in online K-means each data point is typically used only once (or a small number of times) when it arrives.
- **Trade-offs:** Online methods have lower memory and latency but are order-sensitive and may be more noisy; batch methods are more stable per-iteration but require more memory and computation.

(b) (*5 pts*) Show that after each update, the objective function

$$E(\{\mu_k\}) = \sum_{n=1}^{N} \min_k \|x_n - \mu_k\|^2$$

does not increase.

Each online update consists of two steps: (i) assignment — assign the current point x_n to its nearest cluster center, and (ii) update — adjust that cluster's center by the incremental mean rule. We show both steps do not increase the objective.

(1) Assignment step: Assigning x_n to the nearest current center replaces its previous contribution to the objective by a value that is no larger (the minimum distance to the available centers), while other points remain unchanged. Thus the assignment step cannot increase E.

(2) Update step: Let cluster k have current center μ and contain m points S. After adding x we have $S' = S \cup \{x\}$ with $N = m + 1$, and the updated center is

$$\mu' = \mu + \frac{1}{N}(x - \mu),$$

the mean of S'. For any $y \in S'$,

$$y - \mu' = (y - \mu) - \frac{1}{N}(x - \mu),$$

so

$$\|y - \mu'\|^2 = \|y - \mu\|^2 - \frac{2}{N}(y - \mu)^\top (x - \mu) + \frac{1}{N^2}\|x - \mu\|^2.$$

Summing over $y \in S'$ and using $\sum_{y \in S}(y - \mu) = 0$ (since μ is the mean of S) gives

$$\sum_{y \in S'} \|y - \mu'\|^2 = \sum_{y \in S'} \|y - \mu\|^2 - \frac{1}{N}\|x - \mu\|^2,$$

i.e. the total squared error for cluster k decreases by $\frac{1}{N}\|x - \mu\|^2 \geq 0$. Other clusters are unchanged. Combining the non-increase from assignment and the decrease from update, the overall objective $E(\{\mu_k\})$ does not increase after each online update (strictly decreases if $x \neq \mu$).

4 MLE vs MAP

In this problem, we explore the difference between the Maximum Likelihood Estimate (MLE) and the Bayesian Maximum A Posteriori (MAP) estimate. Suppose we have a coin with unknown bias Y. If we flip the coin we observe a Bernoulli random variable:

$$X \sim \textbf{Bernoulli}(Y) \;=\; p(X = x \mid Y) = Y^x(1-Y)^{1-x}$$

The coin will land heads ($X = 1$) with probability Y and tails ($X = 0$) with probability $1 - Y$. We flip the coin (independently) N times to collect the dataset $\mathcal{D} = \{X_n\}_{n=1}^N$. We observe the coin lands heads $N_1 = \sum_{n=1}^N X_n$ times and tails $N_0 = N - N_1$ times.

For this problem, we will also introduce a prior on the probability Y that the coin lands heads. The Beta distribution is a common prior for probabilities. The Beta distribution has the density:

$$p(y \mid \alpha, \beta) = \frac{1}{B(\alpha,\beta)} y^{\alpha-1}(1-y)^{\beta-1}, \quad y \in (0,1),$$

where $\alpha > 0$ and $\beta > 0$ are parameters of the distribution. The function $B(\alpha, \beta)$ is the normalization function

$$B(\alpha,\beta) = \int_0^1 y^{\alpha-1}(1-y)^{\beta-1} dy$$

of the Beta distribution called the Beta function. As a prior we will need to pick values for α and β. These should be based on our beliefs about the coin. Here we will use $\alpha = 3$ and $\beta = 3$:

(a) (*2 pts*) Write the likelihood $p(\mathcal{D}|Y)$ of the data given Y.

Let $N_1 = \sum_{n=1}^N X_n$ be the number of heads and $N_0 = N - N_1$ the number of tails. The likelihood for independent Bernoulli observations is

$$p(\mathcal{D} \mid Y) = \prod_{n=1}^N Y^{X_n}(1-Y)^{1-X_n} = Y^{N_1}(1-Y)^{N_0}.$$

(b) (*2 pts*) Compute the derivative of the log-likelihood function with respect to Y and solve for the MLE estimate for Y.

The log-likelihood is

$$\ell(Y) = \log p(\mathcal{D} \mid Y) = N_1 \log Y + N_0 \log(1-Y).$$

Differentiating and setting to zero:

$$\frac{d\ell}{dY} = \frac{N_1}{Y} - \frac{N_0}{1-Y} = 0 \quad \Rightarrow \quad \hat{Y}_{\text{MLE}} = \frac{N_1}{N}.$$

(c) (*6 pts*) Write the posterior distribution $p(Y|\mathcal{D})$. Express this posterior in the form of a Beta distribution. What are the posterior values of α and β based on the prior values of α and β as well as N_0 and N_1?

With prior $Y \sim \text{Beta}(\alpha, \beta)$, by Bayes' rule

$$p(Y \mid \mathcal{D}) \propto p(\mathcal{D} \mid Y)\, p(Y) = Y^{N_1}(1-Y)^{N_0} \cdot Y^{\alpha-1}(1-Y)^{\beta-1}.$$

Hence the posterior is

$$p(Y \mid \mathcal{D}) = \text{Beta}(\alpha + N_1,\ \beta + N_0),$$

so the updated parameters are

$$\alpha' = \alpha + N_1, \qquad \beta' = \beta + N_0.$$

(d) (*4 pts*) Compute the derivative of the log-posterior and solve for the MAP Estimate.

The log-posterior (ignoring constants) is

$$\ell_{\text{post}}(Y) = (\alpha + N_1 - 1)\ln Y + (\beta + N_0 - 1)\ln(1 - Y).$$

Differentiating and solving gives

$$\frac{\alpha + N_1 - 1}{Y} - \frac{\beta + N_0 - 1}{1 - Y} = 0 \quad \Rightarrow \quad \hat{Y}_{\text{MAP}} = \frac{\alpha + N_1 - 1}{\alpha + \beta + N - 2},$$

where $N = N_1 + N_0$. Note this interior solution is valid when $\alpha + N_1 > 1$ and $\beta + N_0 > 1$; otherwise the MAP may lie at the boundary 0 or 1. For $\alpha = \beta = 3$, $\hat{Y}_{\text{MAP}} = (N_1 + 2)/(N + 4)$.

(e) (*2 pts*) In a 1 sentence, what is the difference between the MLE and MAP estimates? Note: simply putting the your solutions to the previous parts is not sufficient.

MLE maximizes the likelihood using only the observed data, whereas MAP maximizes the posterior and thus combines the likelihood with a prior (equivalently adding a regularization term derived from the prior).

(f) (*5 pts*) How would you increase or decrease the regularization by varying α and β? *Hint: Regularization was covered in the bias variance trade off lectures: more regularization results in a higher bias and lower variance.*

The Beta prior behaves like pseudo-counts: $S = \alpha + \beta$ is the prior strength and $\mu_{\text{prior}} = \alpha/(\alpha + \beta)$ the prior mean. Concretely:

- Increase regularization: increase $\alpha + \beta$ (keeping the ratio $\alpha/(\alpha + \beta)$ fixed) so the prior contributes more weight relative to data; this pulls estimates toward μ_{prior} (higher bias, lower variance).
- Decrease regularization: reduce $\alpha + \beta$ (e.g. set α, β close to 1) so data dominate and MAP approaches MLE (lower bias, higher variance).
- Directional bias: make $\alpha \gg \beta$ to bias toward 1 (heads), or $\beta \gg \alpha$ to bias toward 0 (tails).